Sistemas Distribuidos — Nombrado en SD

Trabajo práctico: "Nombrado en Sistemas Distribuidos" Plataforma distribuida de comercio electrónico.

Dato	Valor
Materia	Sistemas Operativos — Sistemas Distribuidos
Carrera	Licenciatura en Sistemas de Información
Docente	Ing. Mg. Ulises Rapallini

Parte 1 — Respuestas a las preguntas teóricas

1. Nombre, identificador y dirección

Dado el par `inventory-service` (nombre) y `172.18.0.5` (dirección):

1. ¿Qué diferencia existe entre el nombre `inventory-service` y la dirección `172.18.0.5`? El nombre es una etiqueta legible y significativa para humanos que identifica al recurso de forma **lógica**, independiente de dónde esté alojado. La dirección es el identificador **físico** (en este caso una IP) que indica dónde se encuentra el recurso dentro de la red en un momento dado. El nombre es estable; la dirección puede cambiar.

2. ¿Qué se entiende por nombre dentro de un sistema distribuido? Una cadena de caracteres (generalmente legible) que se usa para referirse a una entidad (proceso, servicio, recurso, objeto) de forma independiente de su ubicación física. Permite referenciar un recurso sin conocer su dirección.

3. ¿Qué se entiende por identificador? Un nombre que cumple propiedades estrictas: cada identificador refiere a **una única entidad**, cada entidad es referida por **un único identificador**, y un identificador nunca se reutiliza (no se reasigna a otra entidad, ni siquiera después de que la original desaparezca). Por ejemplo, un UUID o un DNI. Un identificador permite comparar inequívocamente si dos referencias apuntan a la misma entidad.

4. ¿Qué se entiende por dirección? El nombre del punto de acceso físico de una entidad en la red. Indica dónde se puede alcanzar la entidad (por ejemplo, una dirección IP + puerto). Una dirección está atada a la ubicación actual y por lo tanto no es estable en el tiempo.

5. ¿Una dirección puede utilizarse como identificador? Justifique. No de forma correcta. Una dirección (p. ej. una IP) se refiere a la *ubicación*, no a la *entidad*. Si la entidad se mueve, se reinicia o es reemplazada, la dirección cambia o es reasignada a otra entidad, por lo que deja de identificar unívocamente a la original. Además, una misma entidad puede tener varias direcciones (varias NIC) y una dirección puede cambiar de dueño, violando las propiedades de un identificador.

6. ¿Puede una misma entidad tener diferentes nombres? Proporcione un ejemplo. Sí. Un mismo servicio puede ser accedido como `inventory-service` (nombre interno de Docker), `api.inventory.empresa.com` (nombre DNS jerárquico) y `172.18.0.5:8080` (dirección). Todos apuntan a la misma entidad lógica.

7. ¿Puede un mismo nombre representar diferentes instancias de un servicio? Sí. En un esquema de escalado horizontal, el nombre `inventory-service` puede resolver a **varias** direcciones IP (una por instancia/réplica). El nombre se convierte en un nombre lógico de *grupo*, y un balanceador de carga o el sistema de descubrimiento decide a cuál instancia concreta derivar cada petición.

---

2. Resolución de nombres

1. ¿Qué significa resolver un nombre? Es el proceso de traducir un nombre lógico en la información necesaria para localizar la entidad (normalmente una dirección IP y un puerto). Dado `inventory-service`,

resolverlo significa obtener su IP, p. ej. `172.18.0.5`.

2. ¿Qué componente transforma `inventory-service` en una dirección? Un **resolver** (resolutor) o servicio de nombres. En Docker es el **DNS embebido** (servidor en `127.0.0.11`); en Internet es el **DNS**; en Kubernetes es el **CoreDNS/kube-dns** junto al objeto Service. El proceso cliente consulta a este componente y recibe la dirección.

3. ¿Por qué conviene utilizar nombres en lugar de IP configuradas directamente? Porque las IP son inestables: cambian al reiniciar, escalar, migrar o reemplazar una instancia. Si el código usara la IP, habría que modificarlo cada vez que cambia la topología. El nombre desacopla la referencia lógica de la ubicación física.

4. ¿Qué ocurre si `inventory-service` cambia su dirección IP? Nada desde el punto de vista del cliente, siempre que se use el nombre. El servicio de nombres actualiza el registro y las siguientes resoluciones devuelven la nueva IP. Si el cliente hubiera usado la IP directa, quedaría apuntando a una dirección obsoleta y las peticiones fallarían.

5. ¿Qué relación existe entre resolución de nombres y transparencia de ubicación? La transparencia de ubicación es la propiedad por la cual el cliente no necesita saber dónde está el recurso. La resolución de nombres es el mecanismo que la habilita: el cliente usa un nombre estable y el sistema de nombres resuelve dinámicamente la ubicación actual, ocultando los cambios de dirección.

---

3. Espacios de nombres

Dado `api.inventory.empresa.com`:

1. ¿Qué es un espacio de nombres? Es el conjunto de nombres organizados bajo unas reglas de estructura y asignación. Define la sintaxis (cómo se forma un nombre válido) y la semántica (cómo se interpreta) de los nombres de un dominio, incluyendo quién puede crear qué nombres y cómo se resuelven.

2. ¿Corresponde a un esquema plano o jerárquico? Jerárquico. Los nombres se estructuran en niveles separados por puntos (`.`), cada uno dentro de un dominio administrado por un nivel superior.

3. Identifique los diferentes niveles del nombre. - `com` → dominio de nivel superior (TLD). - `empresa` → dominio de segundo nivel (organización). - `inventory` → subdominio (servicio/departamento). - `api` → subdominio más específico (o host) que apunta al punto de acceso de la API.

4. ¿Qué ventajas presenta un espacio de nombres jerárquico? - **Escalabilidad**: se pueden agregar niveles sin límite global. - **Delegación**: cada nivel administra sus subniveles (la raíz delega en `.com`, `.com` en `empresa`, etc.). - **Evita colisiones**: los nombres son únicos globalmente por la jerarquía. - **Localidad y mantenimiento**: los cambios se limitan al subdominio afectado.

5. ¿Por qué DNS utiliza una estructura jerárquica? Porque permite distribuir la responsabilidad de resolución entre muchos servidores autónomos, delegando por niveles (raíz → TLD → dominio). Esto hace al sistema escalable a millones de nombres, tolerante a fallos y administrable de forma descentralizada, imposible con un único registro plano centralizado.

---

4. DNS

1. ¿Qué función cumple DNS? Traducir nombres de dominio legibles (p. ej. `www.google.com`) en direcciones IP utilizables por la red, y viceversa. Es el "directorio" de Internet.

2. ¿Por qué DNS puede considerarse un sistema distribuido? Porque está implementado como una jerarquía de servidores repartidos por todo el mundo (servidores raíz, TLD, autoritativos, recursivos/resolvers), que cooperan entre sí, se replican y delegan zonas. No hay un único punto central que contenga todos los nombres.

3. ¿Qué información almacena un servidor DNS? Registros de recursos (RR): `A/AAAA` (nombre → IP), `CNAME` (alias), `MX` (mail), `NS` (servidor autoritativo de una zona), `PTR` (IP → nombre, inversa), `SOA` (autoridad de la zona), `TXT`, etc., junto con sus TTL (tiempo de vida en caché).

4. ¿Qué diferencia existe entre un nombre de dominio y una dirección IP? El nombre de dominio es una identificación lógica y mnemotécnica de una entidad; la IP es una identificación numérica de su ubicación/interface de red. El nombre se resuelve a una IP mediante DNS. El nombre es más estable y legible; la IP puede cambiar.

5. ¿Qué ventajas proporciona el uso de caché? Reduce la latencia (se evita re-consultar la jerarquía), descarga a los servidores autoritativos, mejora la tolerancia a fallos (respuestas disponibles aunque un servidor esté caído mientras el TTL no expire) y reduce el tráfico de red. Su coste es una eventual inconsistencia temporal mientras no expira el TTL.

6. ¿Qué ocurriría si las aplicaciones tuvieran que conocer directamente las IP de todos los servicios? Sería inviable de mantener: cada cambio de IP obligaría a modificar y redesplegar todas las aplicaciones cliente, no habría tolerancia a reasignaciones ni balanceo, y la administración de un gran número de servicios sería propensa a errores y cortes. El desacople nombre–dirección es lo que hace manejable un sistema distribuido.

---

5. Nombrado en arquitecturas modernas

1. ¿Por qué resulta inconveniente configurar manualmente las IP de los microservicios? Porque en entornos de contenedores/orquestación las IP son efímeras: se asignan al arrancar y cambian al recrear, escalar o migrar una instancia. Mantenerlas a mano no escala y genera configuraciones frágiles que se rompen ante cualquier cambio.

2. ¿Qué ocurre cuando un contenedor es eliminado y creado nuevamente? Se crea una instancia nueva que normalmente recibe una dirección IP distinta (y siempre una identidad nueva). Si otros servicios la referenciaban por IP, quedan apuntando a una dirección obsoleta. Si la referencian por nombre, el sistema de nombres se actualiza y las resoluciones siguen funcionando.

3. ¿Por qué un nombre puede permanecer estable aunque cambie la dirección? Porque el nombre es una abstracción lógica independiente de la ubicación. El servicio de nombres mantiene el vínculo nombre → dirección actual y lo actualiza cuando la entidad cambia de ubicación, de modo que la referencia lógica del cliente no cambia.

4. ¿Qué se entiende por Service Discovery? Es el mecanismo por el cual los servicios se registran (nombre + ubicación) en un registro central y los clientes consultan ese registro para localizar dinámicamente las instancias disponibles. Incluye registro, consulta y, normalmente, chequeo de salud para detectar instancias caídas.

5. ¿Qué relación existe entre Service Discovery y nombrado? El Service Discovery es una capa de nombrado dinámico: el "nombre" del servicio es la clave estable y el registro mantiene la lista actual de direcciones asociadas. Es la evolución del concepto clásico de resolución de nombres aplicada a servicios efímeros y replicados.

6. ¿Qué diferencias existen entre DNS tradicional y descubrimiento de servicios? | Aspecto | DNS tradicional | Service Discovery | |—|—|—| | Objetivo | Resolver nombres de hosts/dominios | Localizar instancias de un servicio | | Dinamismo | Bajo (registros relativamente estáticos, TTL) | Alto (instancias entran/salen continuamente) | | Salud | No conoce el estado del destino | Incorpora health checks | | Resultado | Una IP (o varias) | Lista de instancias vivas + metadatos/puertos | | Actualización | Por propagación de zonas y TTL | En tiempo real ante registro/baja |

Parte 2 — Ejercicios prácticos

Ejercicio 1 — Crear una red Docker

```
docker network create ecommerce-network
docker network ls
docker network inspect ecommerce-network
```

Salida real:

```
$ docker network create ecommerce-network
9bc44736e3edc1b6fbaeb23fd04b107ab6dd6e21d59bd14c64666af96562e814

$ docker network ls
NETWORK ID     NAME               DRIVER    SCOPE
9bc44736e3ed   ecommerce-network  bridge    local
...

$ docker network inspect ecommerce-network
{
    "Name": "ecommerce-network",
    "Driver": "bridge",
    "IPAM": { "Config": [ { "Subnet": "172.22.0.0/16", "Gateway": "172.22.0.1" } ] }
}
```

Análisis: 1. **¿Qué función cumple una red Docker?** Crea un espacio de red aislado (tipo *bridge*) al que se conectan los contenedores, permitiéndoles comunicarse entre sí por su nombre y asignándoles direcciones dentro de la subred. 2. **¿Qué servicios pueden comunicarse en ella?** Todos los contenedores que se conecten a `ecommerce-network` pueden alcanzarse entre sí (comunicación directa). Los que no estén en la red no. 3. **¿Qué relación tiene con la resolución de nombres?** Docker asocia a esta red un **DNS embebido** que registra automáticamente cada contenedor con su nombre. Por eso dentro de la red el nombre `inventory-service` se resuelve a su IP sin configuración manual.

Ejercicio 2 — Crear el servicio de inventario

```
docker run -d --name inventory-service --network ecommerce-network nginx
docker ps
docker inspect inventory-service
```

Salida real:

```
$ docker inspect -f "Nombre: {{.Name}} | IP: ... | Estado: {{.State.Status}}" inventory-service
Nombre: /inventory-service | IP: 172.22.0.2 | Estado: running
```

Registro: nombre `inventory-service`, IP `172.22.0.2`, red `ecommerce-network`, estado `running`.

Análisis de la diferencia nombre vs. dirección: el nombre `inventory-service` es la etiqueta lógica estable; la IP `172.22.0.2` es la ubicación física efímera asignada en este arranque.

Ejercicio 3 — Crear un cliente

```
docker run -it --name orders-service --network ecommerce-network alpine
# dentro del contenedor:
apk add --no-cache bind-tools curl
ping inventory-service
```

```
nslookup inventory-service
getent hosts inventory-service
```

Salida real:

```
$ ping -c 3 inventory-service
PING inventory-service (172.22.0.2): 56 data bytes
64 bytes from 172.22.0.2: ... time=2.536 ms
--- inventory-service ping statistics ---
3 packets transmitted, 3 packets received, 0% packet loss

$ nslookup inventory-service
Server:   127.0.0.11
Address:  127.0.0.11#53
Name:   inventory-service
Address: 172.22.0.2

$ getent hosts inventory-service
172.22.0.2        inventory-service

$ cat /etc/resolv.conf
nameserver 127.0.0.11
```

Análisis: - **¿Quién resolvió el nombre?** El **DNS embebido de Docker**, que escucha en `127.0.0.11` (según `/etc/resolv.conf`). Es él quien devuelve `172.22.0.2` para `inventory-service`. - **¿Fue necesario conocer la IP?** No; el usuario/cliente solo usó el nombre. - **¿Qué ventaja presenta?** Desacopla la referencia de la ubicación: si la IP cambia, el nombre sigue resolviendo correctamente.

Ejercicio 4 — Cambio de dirección

```
docker stop inventory-service
docker rm inventory-service
docker run -d --name inventory-service --network ecommerce-network nginx
docker inspect inventory-service
# desde orders-service:
nslookup inventory-service
```

Salida real (forzando que la IP cambie ocupando direcciones intermedias):

```
Antes:  inventory-service -> 172.22.0.2
Después de recrear: inventory-service -> 172.22.0.5

$ nslookup inventory-service   (desde orders-service, SIN cambios)
Name:   inventory-service
Address: 172.22.0.5
```

Respuestas: 1. **¿Cambió la IP?** Sí, de `172.22.0.2` a `172.22.0.5`. 2. **¿Cambió el nombre?** No, sigue siendo `inventory-service`. 3. **¿Fue necesario modificar el cliente?** No; `orders-service` sigue resolviendo `inventory-service` y obtiene la nueva IP automáticamente. 4. **¿Qué relación tiene con transparencia de ubicación?** Es un ejemplo directo: el cliente usa un nombre lógico estable y el cambio físico de ubicación (IP) le resulta transparente.

Ejercicio 5 — DNS tradicional

```
nslookup www.google.com
```

Salida real (desde `orders-service`):

```
$ nslookup www.google.com
Server:   127.0.0.11
Name:   www.google.com
Address: 142.251.150.119
Address: 142.251.154.119
Address: 142.251.151.119
...
```

Salida real (desde el host):

```
Servidor:  Docsis-Gateway.fibertel.com.ar
Address:  192.168.0.1
Nombre:  www.google.com.com.ar
Address:  64.31.3.235
```

Comparación DNS de Internet vs. resolución interna de Docker: - En Internet, la resolución pasa por la jerarquía DNS (resolver del ISP/router `192.168.0.1` → servidores recursivos → autoritativos), y `www.google.com` devuelve **múltiples IP** (replicación + balanceo global). - Dentro de Docker, el resolver embebido (`127.0.0.11`) responde localmente por los contenedores de la red, y reenvía al DNS externo el resto de dominios. Ambos comparten el mismo principio: un nombre lógico se resuelve a una o varias direcciones.

Ejercicio 6 — InventoryService en C

Proyecto `InventoryService/` (Web API .NET 8). Controlador `Controllers/InventoryController.cs` que expone `GET /api/inventory/{productId}`:

```
[ApiController]
[Route("api/[controller]")]
public class InventoryController : ControllerBase
{
    [HttpGet("{productId}")]
    public IActionResult Get(string productId)
    {
        return Ok(new
        {
            ProductId = productId,
            Stock = 25,
            Server = Environment.MachineName
        });
    }
}
```

Prueba real (contenedor corriendo):

```
GET http://localhost:8080/api/inventory/P001
{"productId":"P001","stock":25,"server":"4910dfada409"}
```

El campo `server` devuelve `Environment.MachineName` (el ID corto del contenedor), útil para distinguir instancias en escalado.

Ejercicio 7 — OrdersService

Proyecto `OrdersService/` cuyo controlador consume el inventario **por nombre lógico** (`inventory-service`), no por IP:

```
var client = new HttpClient();
var response = await client.GetAsync(
    "http://inventory-service:8080/api/inventory/P001");
var content = await response.Content.ReadAsStringAsync();
```

Implementado como endpoint `GET /api/orders/{orderId}` con `AddHttpClient()`.

Prueba real:

```
GET http://localhost:8081/api/orders/O001
{"orderId":"O001","inventory":"{\"productId\":\"P001\",\"stock\":25,\"server\":\"4910dfada409\"}","serve
```

Comparación IP directa vs. nombre lógico: con IP directa, un cambio de dirección rompe el cliente y exige modificarlo; con el nombre lógico, el cliente sigue funcionando sin importar la IP actual de `inventory-service`.

Ejercicio 8 — Dockerizar los servicios

Se crearon los `Dockerfile` (multi-etapa: SDK para compilar, `aspnet` para ejecutar) de ambos servicios, más `.dockerignore`. Ver `InventoryService/Dockerfile` y `OrdersService/Dockerfile`.

Build real (exitosa):

```
Image nombradosd-inventory-service Built
Image nombradosd-orders-service Built
```

Ejercicio 9 — Docker Compose

`docker-compose.yml` define los dos servicios en la red `ecommerce-network` (driver bridge), con `depends_on` de `orders-service` hacia `inventory-service`.

```
docker compose up --build
docker compose ps
```

Salida real:

```
$ docker compose ps
NAME                              SERVICE             STATUS          PORTS
nombradosd-inventory-service-1    inventory-service   Up              0.0.0.0:8080->8080/tcp
nombradosd-orders-service-1       orders-service      Up              0.0.0.0:8081->8080/tcp
```

Análisis (nombres, direcciones, resolución): Compose crea la red `nombradosd_ecommerce-network` y registra en el DNS embebido los alias `inventory-service` y `orders-service`. Una consulta dentro de la red resuelve `inventory-service → 172.23.0.2` sin configuración extra.

```
$ nslookup inventory-service   (contenedor auxiliar en la red)
Name:   inventory-service
Address: 172.23.0.2
```

Ejercicio 10 — Recrear la infraestructura

```
docker compose down
docker compose up -d
docker inspect <nombre-del-contenedor>
```

Salida real (antes vs. después):

```
ANTES:     inventory-service -> 172.23.0.2   orders-service -> 172.23.0.3
DESPUÉS:   inventory-service -> 172.23.0.2   orders-service -> 172.23.0.3
```

Determinación: las IP coincidieron (al recrear la red desde cero el asignador las repartió igual), pero los contenedores son **instancias nuevas** (cambió el `server`/hostname, p. ej. de `4910dfada409` a `70d3d8fbf11b`). La conclusión central es que **el cliente no necesita ninguna modificación** cuando cambia la infraestructura:

```
GET http://localhost:8081/api/orders/0002  (mismo código, nueva infraestructura)
{"orderId":"0002","inventory":"{...\"server\":\"70d3d8fbf11b\"...}","server":"47b22aed42ee"}
```

El nombre `inventory-service` se resolvió automáticamente a la nueva instancia.

Ejercicio 11 — Múltiples instancias

Al escalar `inventory-service` a tres réplicas detrás del mismo nombre:

1. **¿Debería OrdersService conocer las tres IP?** No. Solo debe conocer el **nombre** del servicio.
2. **¿Qué problemas generaría?** Conocer y gestionar manualmente tres IP acopla al cliente a la topología: si una réplica cae, cambia de IP o se agrega otra, el cliente debe actualizarse y el reparto de carga no se distribuye bien.
3. **¿Qué mecanismo debería localizar las instancias?** Un sistema de **Service Discovery / balanceador de carga** que, a partir del nombre, devuelva la lista de instancias y distribuya las peticiones.
4. **¿Cómo se relaciona con balanceo de carga?** El nombre resuelve a un punto de entrada que reparte el tráfico entre las réplicas (round-robin, aleatorio, etc.). El cliente ve un único nombre; el balanceo es transparente.
5. **¿Qué ocurre si una instancia falla?** El balanceador/service discovery la detecta (health checks) y deja de enviarle tráfico, dirigiendo las peticiones a las réplicas sanas. El cliente no se entera.

Ejercicio 12 — Service Discovery

1. **¿Qué información mantiene?** La lista de instancias vivas de cada servicio: nombre del servicio, direcciones (IP+puerto), metadatos y estado de salud.
2. **¿Cómo se registran nuevas instancias?** Cada instancia, al arrancar, se **auto-registra** en el registro (o un agente lo hace por ella) y mantiene un *heartbeat*; al detenerse se da de baja.
3. **¿Cómo se detectan instancias no disponibles?** Mediante **health checks** periódicos y/o expiración del *heartbeat*: si una instancia no responde, se elimina del registro.
4. **¿Qué relación existe con nombrado?** Es nombrado dinámico: el nombre del servicio es la clave estable y el registro mantiene el mapeo nombre → conjunto actual de direcciones.
5. **¿Por qué es importante en microservicios?** Porque los microservicios se crean, destruyen y escalan continuamente; el discovery permite que los clientes siempre localicen instancias sanas sin configuración estática.

Ejercicio 13 — Kubernetes

1. **¿Qué problema de nombrado resuelve Kubernetes Service?** Da una **IP virtual estable y un nombre DNS** a un conjunto de Pods efímeros. Los clientes usan el nombre del Service, no las IP de los Pods.
2. **¿Por qué no conviene referenciar directamente las IP de los Pods?** Porque los Pods son efímeros: al reiniciarse, escalarse o reprogramarse cambian de IP y de nombre.
3. **¿Qué ocurre si un Pod es reemplazado?** El Service (vía selectors de etiquetas) actualiza sus *endpoints* para incluir al nuevo Pod y excluir al viejo, manteniendo la misma IP/alias del Service.
4. **¿El cliente necesita conocer el cambio?** No. Sigue usando el nombre del Service (`inventory-service`) y el DNS del clúster (`kube-dns`/CoreDNS) resuelve a la IP estable del Service.

5. **¿Cómo se relaciona con transparencia de ubicación?** El Service actúa como capa de indirección que oculta la ubicación y rotación de los Pods; el cliente se referencia por nombre y no percibe los reemplazos.

Ejercicio 14 — Comparación

Mecanismo	Nombre utilizado	Quién resuelve	Destino	¿Puede cambiar la IP?
DNS Internet	`www.ejemplo.com`	DNS	Servidor	Sí
Docker	`inventory-service`	DNS interno Docker (`127.0.0.11`)	Contenedor	Sí
Kubernetes	`inventory-service`	DNS / Service	Pods	Sí
IP directa	`172.18.0.5`	No existe resolución	Host/Contenedor	La referencia queda obsoleta

(Verificado en la práctica: en Docker el nombre se mantuvo y la IP pasó de `172.22.0.2` a `172.22.0.5` sin tocar el cliente.)

Ejercicio 15 — Conclusión

`NOMBRE → RESOLUCIÓN → DIRECCIÓN → ENTIDAD / SERVICIO`

Los **nombres** son el punto de referencia estable y desacoplado de la ubicación. La **resolución** es el mecanismo que traduce ese nombre en la **dirección** actual de la **entidad** en cada momento. Esta cadena de indirección es la que hace viable un sistema distribuido.

Los componentes de un sistema distribuido **cambian de ubicación, se reinician, escalan horizontalmente y son reemplazados** constantemente; su dirección física es, por tanto, efímera. Si los clientes codificaran direcciones IP, cada cambio obligaría a modificarlos y redesplegarlos, y el sistema se volvería frágil e inmanejable.

Al referenciar por **nombre lógico** (`inventory-service`) y delegar la resolución en un sistema de nombres (DNS, DNS embebido de Docker, Service Discovery, Kubernetes Service), los clientes quedan aislados de esos cambios: la dirección puede variar, la identidad lógica permanece. Esto habilita:

- **Transparencia de ubicación**: el cliente no sabe ni necesita saber dónde está el recurso.
- **Escalabilidad**: muchas instancias comparten un único nombre detrás de un balanceador.
- **Movilidad**: migrar o recrear una entidad no rompe a sus consumidores.
- **Tolerancia a fallos**: el discovery deja de enrutar a instancias caídas sin intervención del cliente.

En síntesis, en un sistema distribuido los **nombres** son la interfaz estable que permite que la **ubicación** sea un detalle dinámico; la **resolución de nombres** es la pieza que conecta ambos mundos, y por eso constituye un pilar fundamental del diseño de sistemas distribuidos.